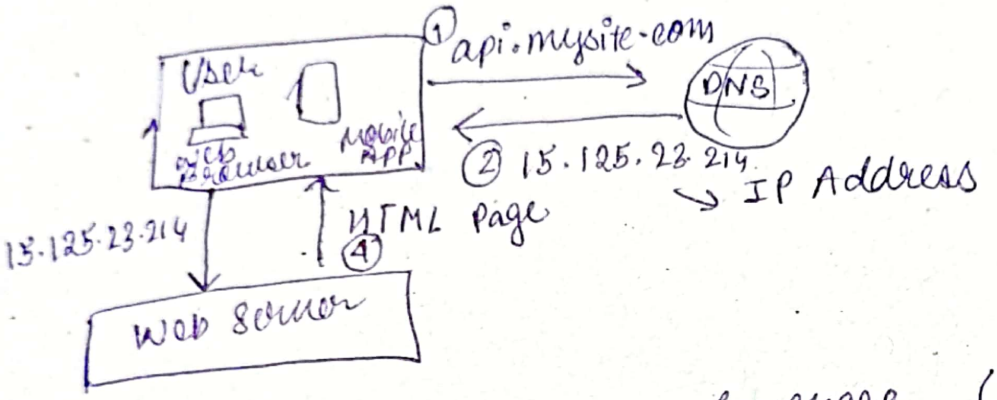
Single Server Setup

- Build a single user to millions of user.

Domain Name :- www.letcode.com
IP Address :- 15.125.23.214



Traffic
- Web Applications
- Mobile APPS

Web Applications - Server side Language (Java, Python etc)
+ client side Language (HTML, JS)

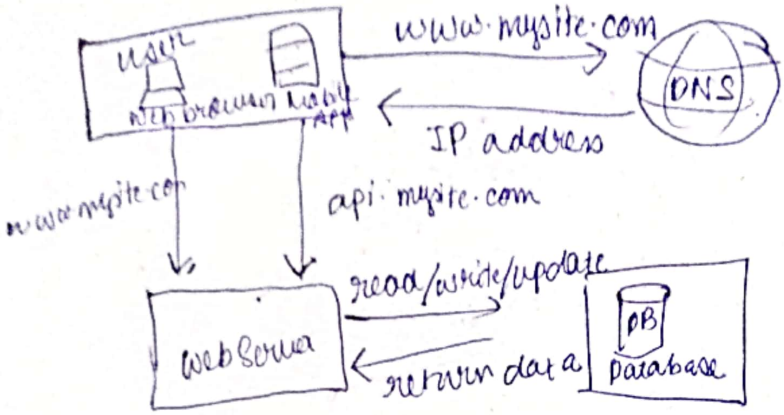
Mobile APPS :- HTTP Protocol with server
API response = JSON

But with growth of users 1 server is not enough.
We need more servers. ⇒ Scale

Server-1
Traffic
(from web apps, mobile apps)

Server-2
Database

Scanned with OKEN Scanner



Which Database to use?

i) Relational Database Non-relational DB.

Relational DB	Non-Relational DB
- RDBMS or SQL Database	- NOSQL Databases
Eg - MYSQL, Oracle, PostgreSQL	- Eg - Cassandra, Amazon DynamoDB
- Represents data in tables & rows	- Grouped into 4 categories • Graph stores • Key value stores • Column stores • Document stores
- Can perform JOIN operations using SQL on different tables.	- JOIN operation not supported.

Which one to use?

i) For most developers - Relational DB

ii) But more are many cases in which Relational DB is not suitable.

iii) when to think about non-relational DB?

a) Your application requires super low latency.

b) When your data is unstructured

c) You only need to serialize & deserialize data (JSON, XML, YAML etc)

d) When you need to store MASSIVE amount of data.

Scanned with OKEN Scanner

Vertical Scaling

↳ Scale-up

↳ Adding more power to your servers

Eg - CPU, RAM

Horizontal Scaling

↳ "Scale-out"

↳ Adding more servers

When to use vertical & when horizontal ?

Vertical Scaling	Horizontal Scaling
• When traffic is LOW.	• For large scale applications
• Simplicity in its advantage	• Huge traffic

Limitations of vertical Scaling

- Impossible to add unlimited CPU & RAM to a single server.
- NO Backup (failover) - If one server goes down. The website / service goes down completely.

That's why for large applications horizontal scaling is preffered due to limitations of vertical scaling.

Load Balancer

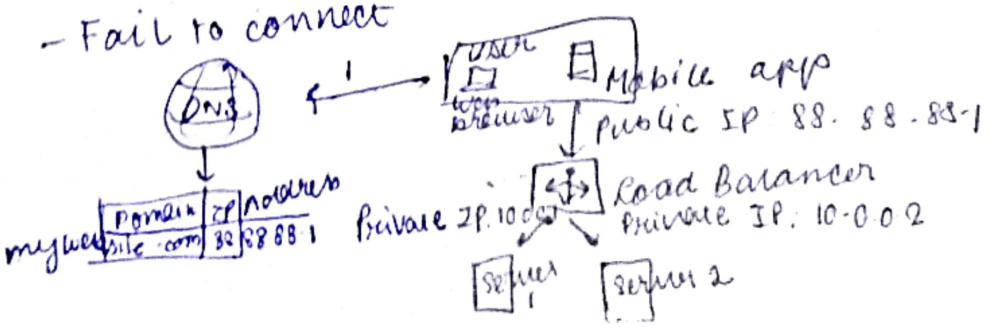
•) Previous Design

•) User connected directly

•) When server is offline

•) Also, when many users access at the same time

- web server load limit
- slow response
- Fail to connect



Scanned with OKEN Scanner

" Evenly distribute web traffic
- users connect to Load Balancer via Public IP

- Load Balancer communicates with web servers via Private IP.

Load Balancer + 2nd web server

- No failover
- Improved availability

- If server1 goes offline, server 2 will get all traffic - so server won't go offline
- we'll add new healthy web server
  - Balance the load
- If website grows rapidly, 2 servers not enough to handle traffic.
- we add one more server and load balancer automatically starts to send requests to them.

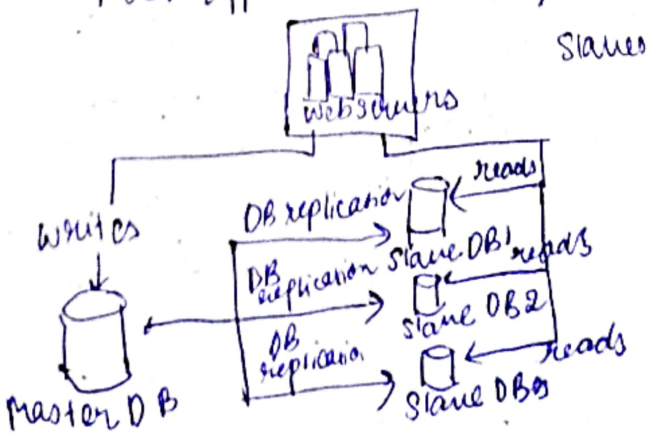
When Database server fails?

Database replication
↳ Master - original DB
↳ Slave - copy (replica)

Master
↳ only write operations
↳ insert, delete, update
(data-modifying commands)

Slave
↳ Read operations.
↳ Gets copies of data from master.

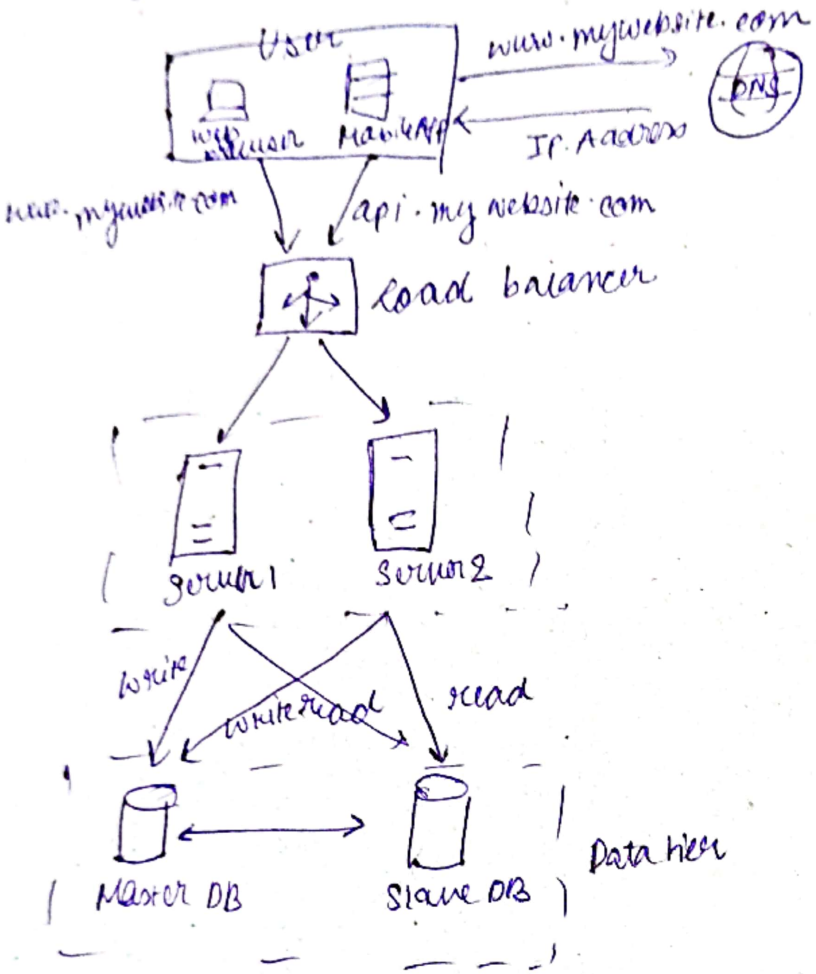
Most applications = reads / writes ↑ high ratio
slaves > masters



Scanned with OKEN Scanner

Advantages of DB Replication

- Better performance
- Reliability
- High Availability

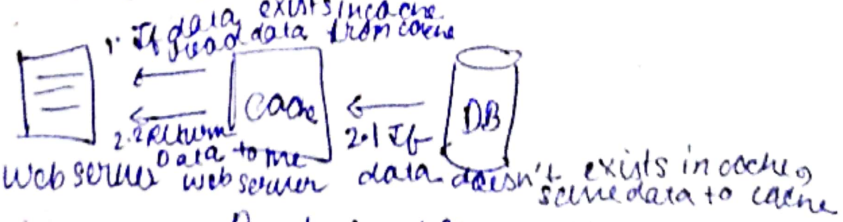


Caching

load / response ↑

cache
- Temporary storage.
- Stores
  - ↳ Results of expensive responses
  - ↳ Frequently accessed data

We can serve requests more quickly.



Read-through cache

Interacting with cache
↳ Simple
↳ cache servers provide APIs

Eg:
```
cache.set('my-key', 'value', 3600 seconds);
cache.get('my-key');
```

Things to consider for using cache?
- Data read frequently + modified infrequently
- Expiration Policy:
  - ↳ Not too low - Data reload from DB.
  - ↳ Not too high - Stale Data

Consistency: Between DB & cache
- Multiple cache servers across different data centers to avoid SPOF → Single point of Failure

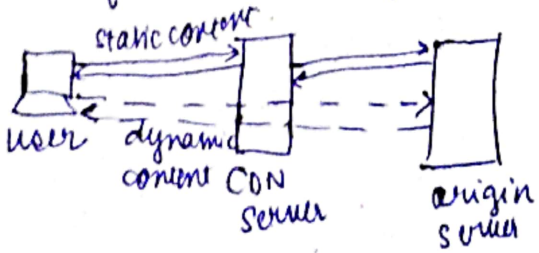
Eviction Policy - when cache gets full
- ↳ LRU
- ↳ LFU
- ↳ FIFO

CDN (Content Delivery Network)

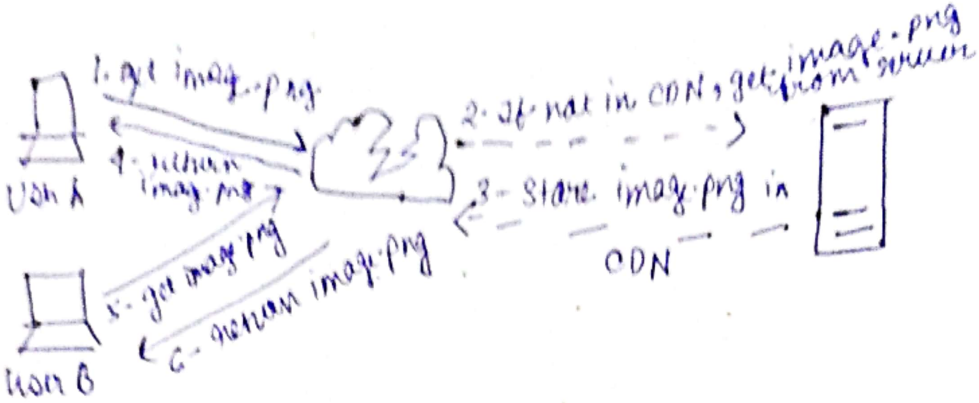
Website → Static contents (images, videos, CSS, JS etc)
Website → Dynamic contents (changes according to user input)

Eg:- Website for selling e-book → static contents

How CDN works (high-level view):-
- User visits a website.
- CDN server closest to user - static content
- If CDN server far = slow website loading

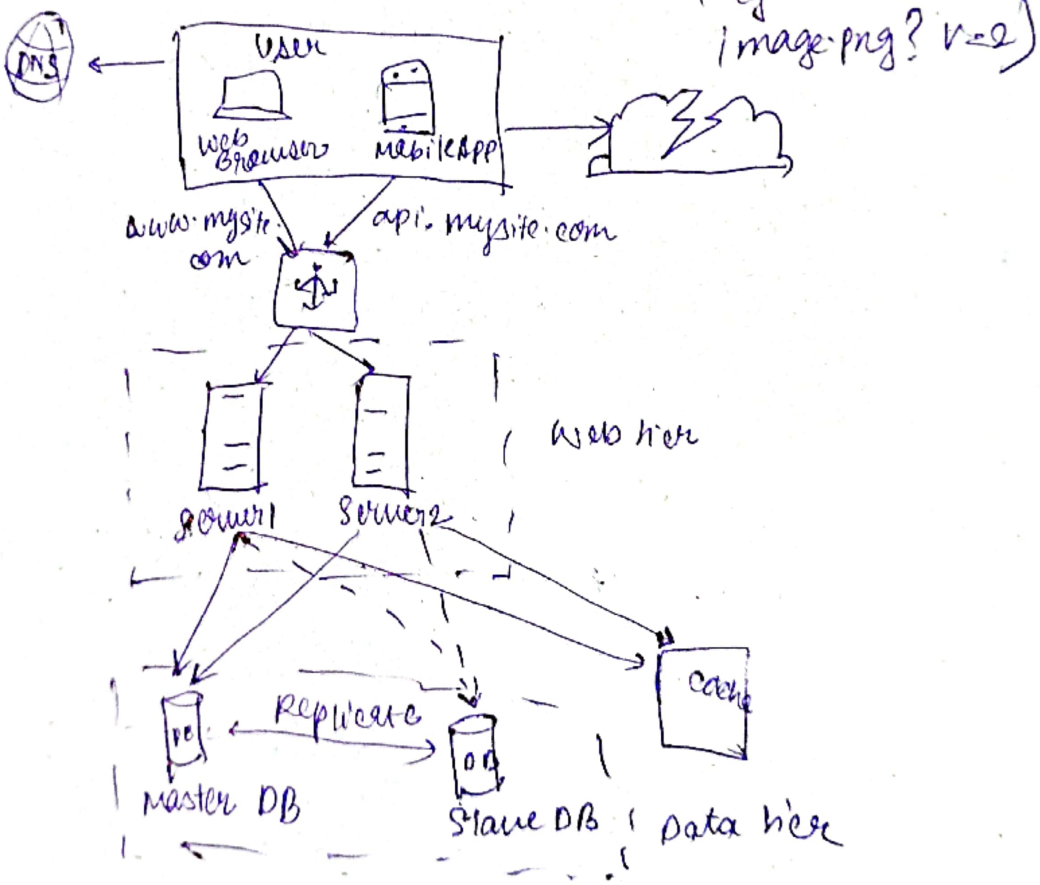


Scanned with OKEN Scanner



Things to consider for using CDN

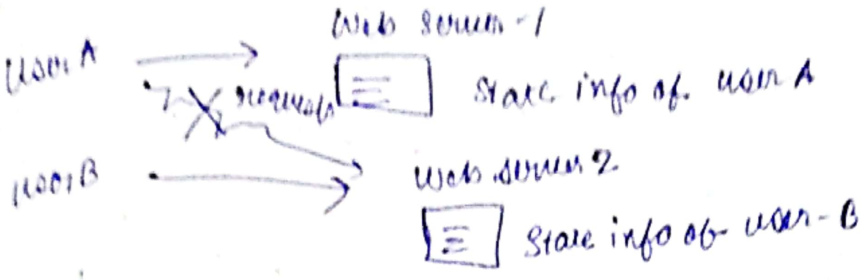
- Cost – Get rid of infrequently used contents
- Expiry time – Not too low, not too high
- CDN Fallback/Backup – Backup when CDN fails
- Invalidating files → API (CDN vendors)
  → Versioning (Eg - image.png?v=2)

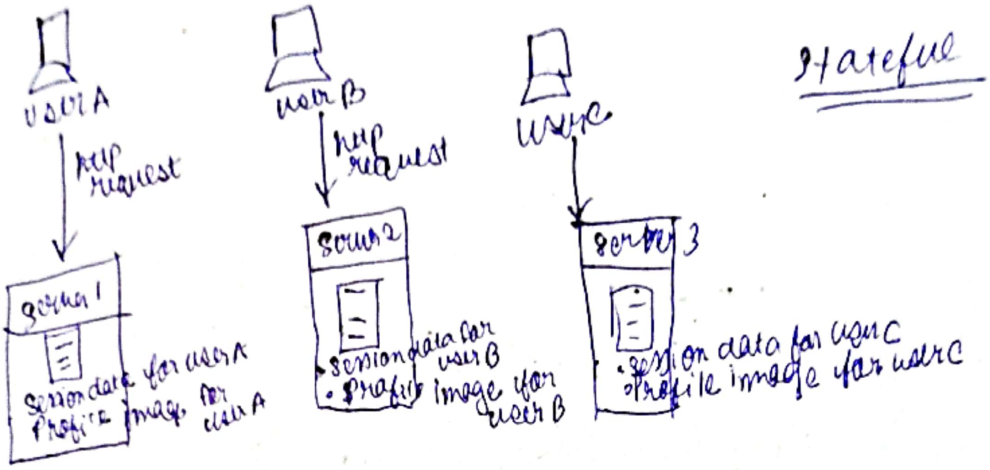


Stateless & Stateful Web Architecture

Session Data/State :- Data that represents user's session on web.
- Identify a user
- Authenticate the user

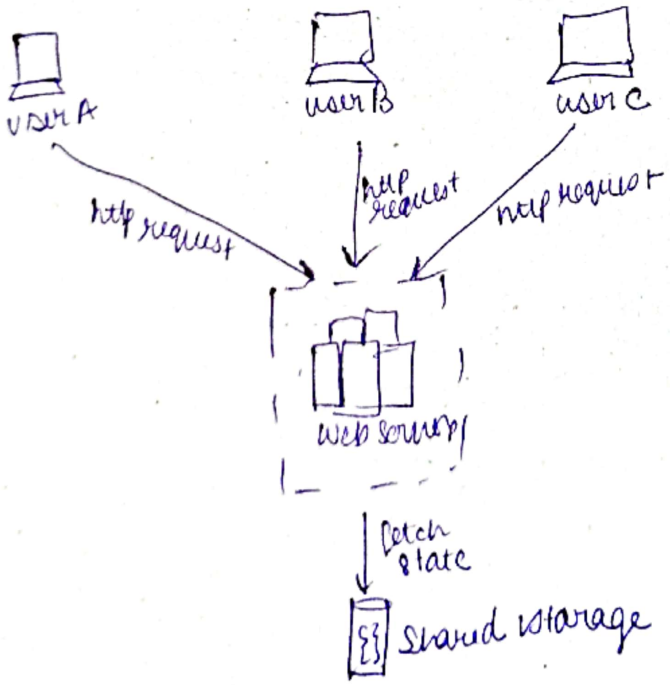
Scanned with OKEN Scanner





Stateful

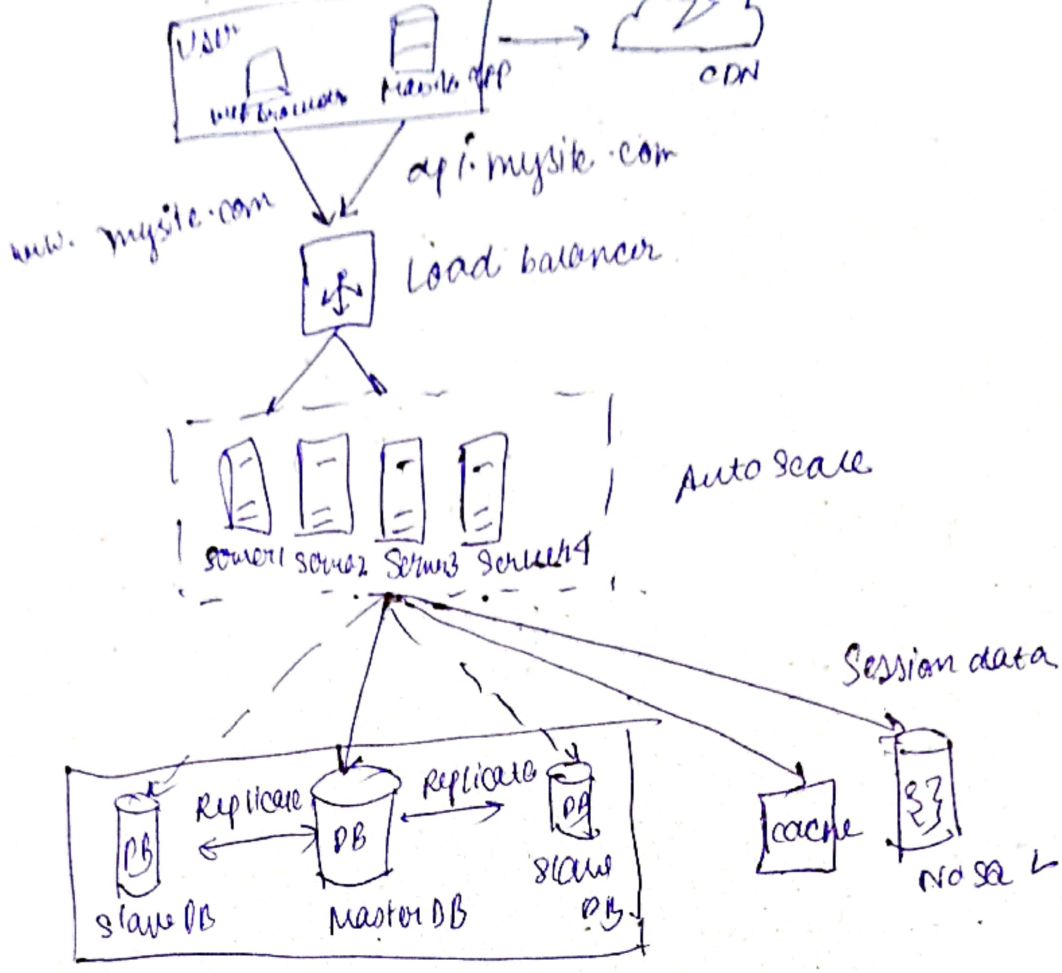
Stateless Web	Stateful web
• Does not remember client data.	• Remembers client data
• Does not store state information.	• stores state information
• simple, robust	• overhead, not simple



New aur service hoga AMAZINO

- Multiple Servers
- Load Balancer
- Master - slave DB
- cache DB
- CDN

Stateless web (shared storage)



Data Centers

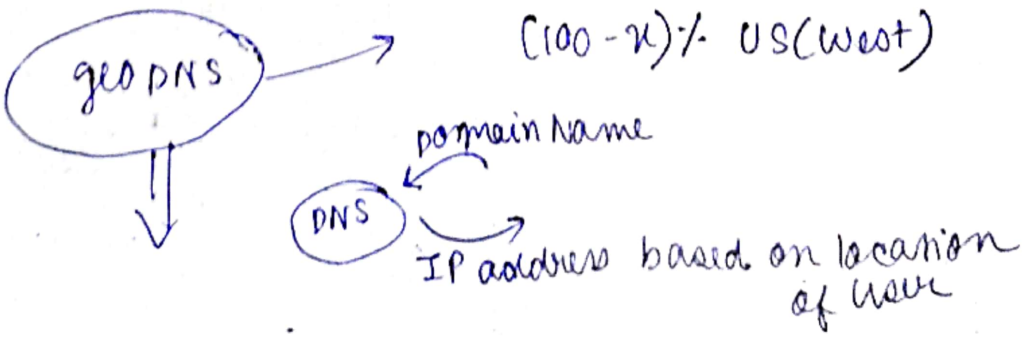
Your website
- Very popular
- Lots of user internationally
- Around the world

To improve availability & better user experience across wider geographical areas we should support multiple Data centers

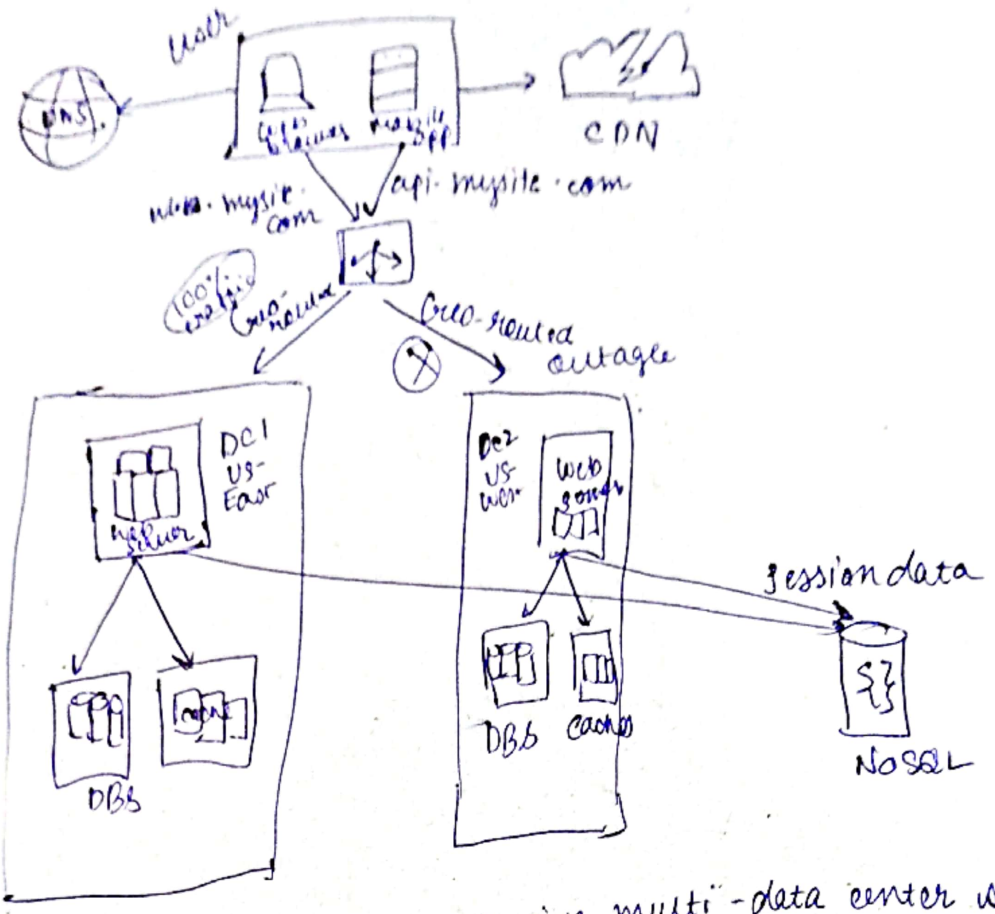
Route to the closest data center.

Example :- US (east) Data center - 1
US (west) Data center - 2

Split traffic :- x % US (East)
(100 - x) % US (West)



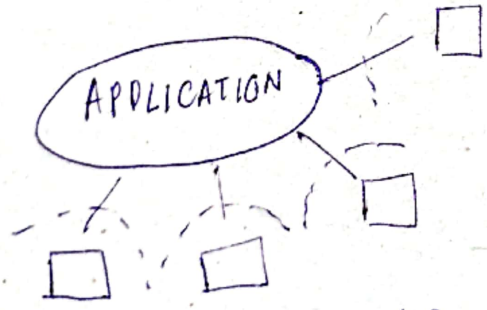
Scanned with OKEN Scanner



Technical challenges to achieve multi-data center setup :-

- Traffic redirection :- geoDNS
- Data synchronization :- Replicate data across multiple data centers.
- Test & Deployment :- Test your website/app at different locations.

Message Queue

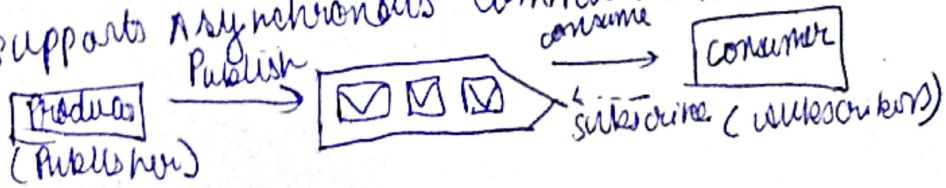


Smaller independent Building blocks

Easier to :- develop
deploy
maintain

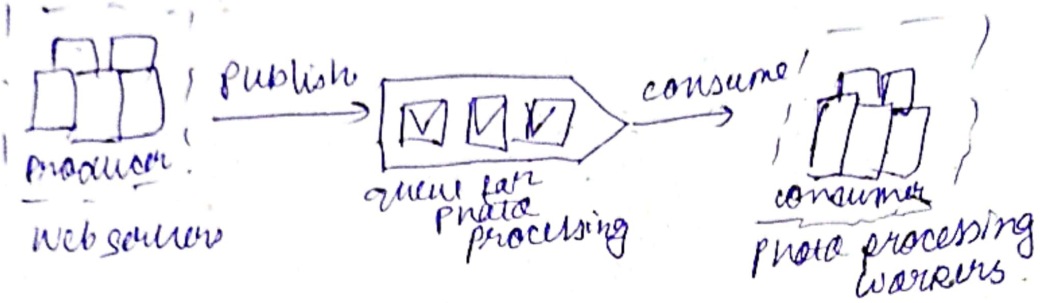
Message queue provide communication & coordination between mem.

- Supports Asynchronous communication.



Scanned with OKEN Scanner

- Producer can produce a message to queue even when consumer is unavailable.
- Consumer can read message from queue even when producer is unavailable.



↳ Independent
↳ Scaled independently
↳ Queue size large ↑
add more consumers/workers
↳ If Queue is empty mostly
reduce consumers/workers

Logging, Metrics, Automation

Small website → Few servers → Not necessity
Busy & popular website → Essential.

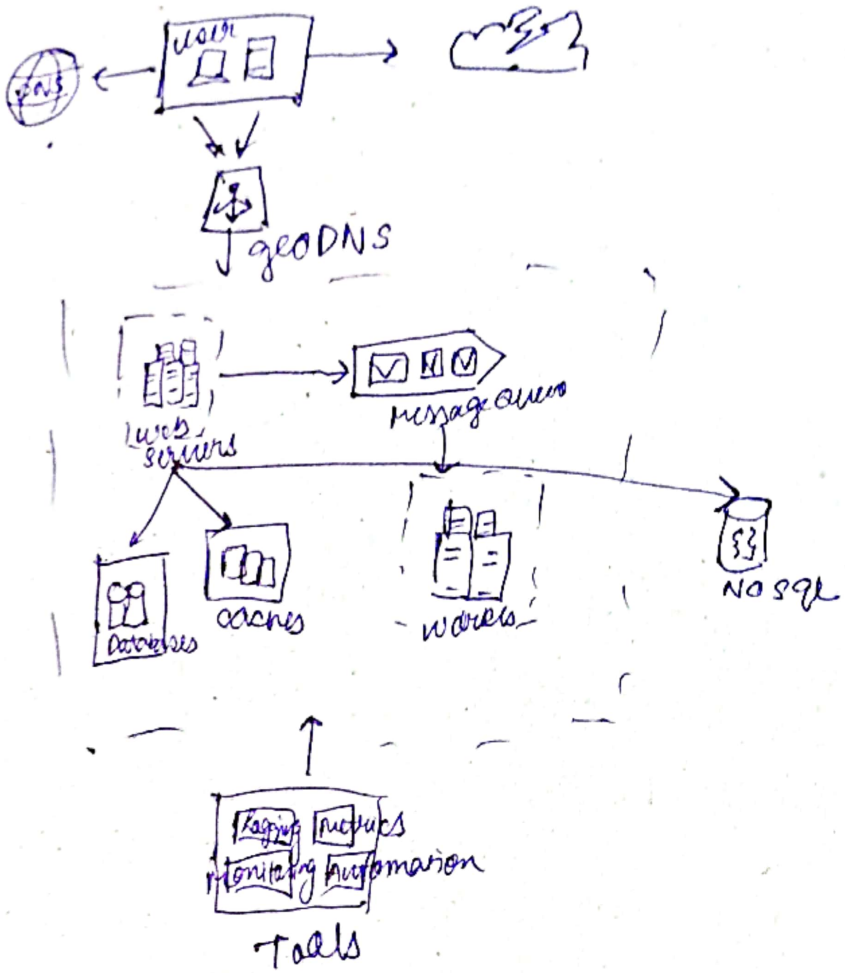
Logging — Monitoring error logs
- Per server level
- Centralized logging
↳ Eg: NewRelic

Metrics - Gain business insights
Aware of health of our service
- Host level metrics: CPU, memory etc.
Eg - Grafana

- Aggregated level metrics: -
Performance of DB, cache etc
- Key Business metrics - Daily active users, revenue etc.

Scanned with OKEN Scanner

Automation :- when system gets big & complex
- Automation tools
- Improve productivity
- Continuous integration
  code check-in verified through automation
- Automating builds, test, deployment etc. Eg. Jenkins, Jobs



Database Scaling, Sharding & Justin Beiber Problems

Data grows ↑ ⇒ Database overloaded ⇒ Scale your DB servers

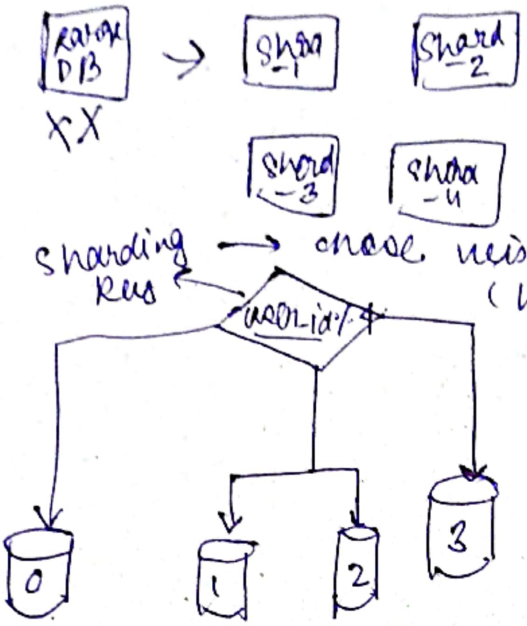
Database Scaling
1) Vertical Scaling
2) Horizontal Scaling

Vertical Scaling: - More power (CPU, RAM, Disk etc.)
- RDS (Amazon Relational DB Service)
  24TB of RAM
- 2013 stackoverflow - (10 million unique visitors everyday) with only one master DB.

Drawbacks: -
- Hardware limits exist
- Single point of Failure
- costly

Scanned with OKEN Scanner

Horizontal Scaling: Sharding (Adding more servers)



Shards - Smaller & easy management

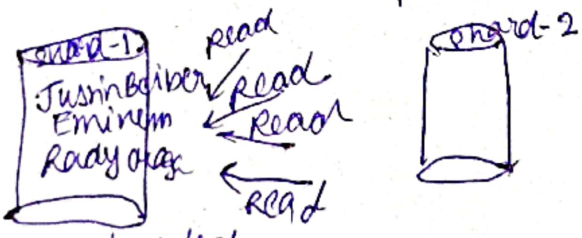
→ chose wisely (very important for even distribution of data)

Schema in the shards are same but data is different.

Drawbacks of sharding

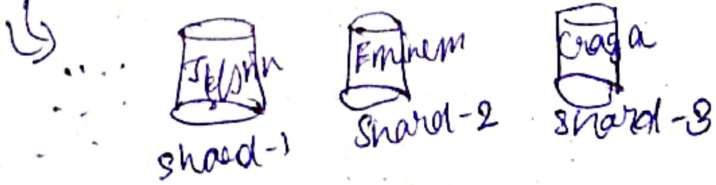
- Resharding → when a shard can no longer hold more.
  ↳ shard exhaustion due to uneven distribution.
  It will require updating sharding function & then move data around.

- Celebrity Problem (Hotspot Key Problem)
  ↳ Excessive access to a particular shard cause shard overload.



overloaded

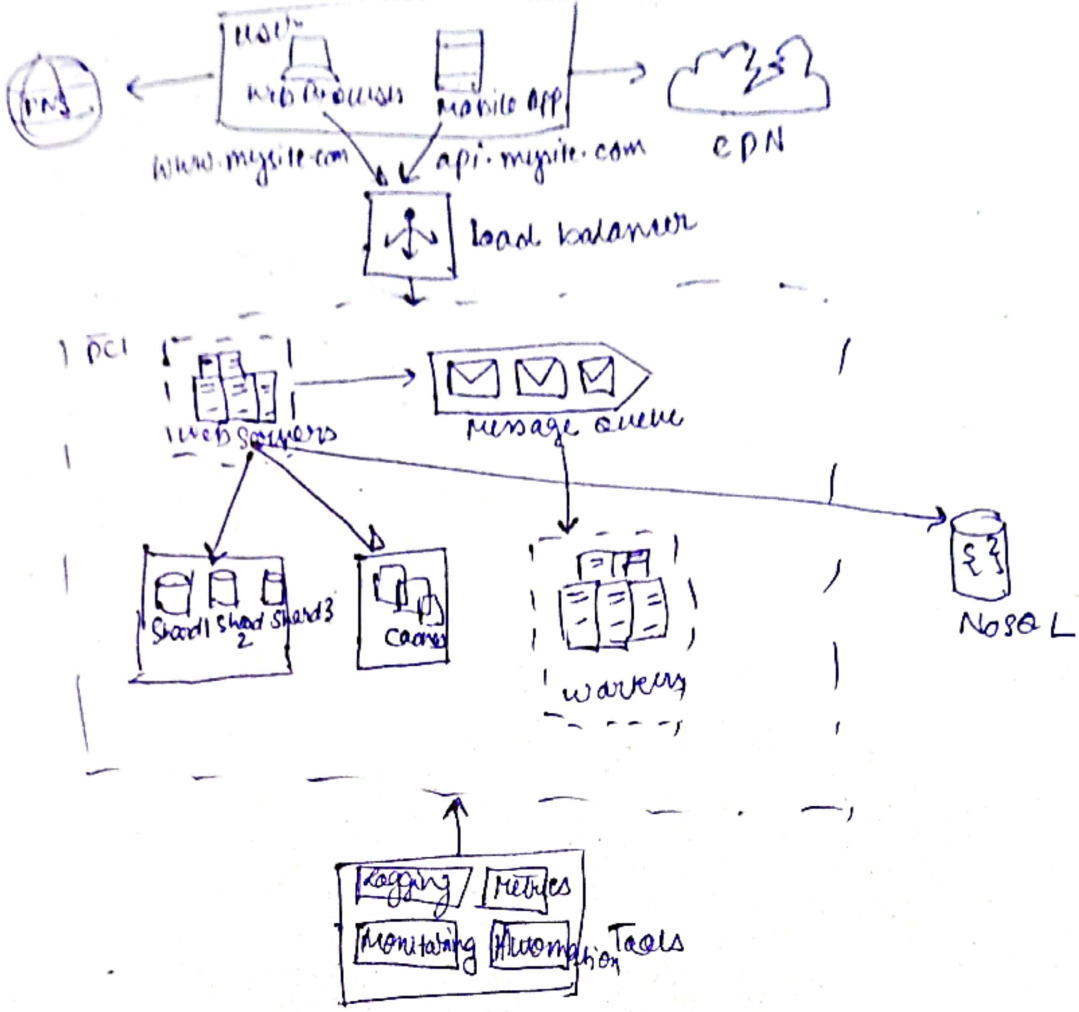
Assign one celebrity to each shard



↳ Sharding makes it difficult to perform JOIN operations.

Common fix - denormalize the database so that queries can be done in a single table.

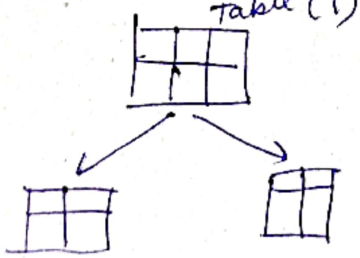
Scanned with OKEN Scanner



Normalization

- Breaking down large tables into smaller & related tables & establishing relationships between them.
- Minimizes redundancy & simplify.

Don't do over normalization.



Employee ID	EmployeeName	Department	Department Location
1	Alice	HR	Building A
2	Bob	IT	Building B
3	Carol	HR	Building A

Normalize the table

Employee ID	EmployeeName
1	Alice
2	Bob
3	Carol

Department
HR
IT
HR

Scanned with OKEN Scanner

Department Table:

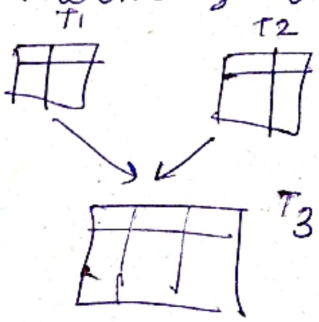
Department	Department Location
HR	Building A
IT	Building B

Find the Department location of EmpID = 2

Select E.EmployeeID, E.EmployeeName, D.DepartmentLocation
From Employee E JOIN Department D on E.Department =
D.Department where E.EmployeeID = 2;

Denormalization

- Deliberately introducing redundancy into a DB table.
- opposite of Normalization.



Why Denormalization?

1) Improved query performance. (No joins)
2) Simplified query.
3) Reduce joins
4) Read intensive scenario → Best

Cons of Denormalization

1) Redundancy → memory waste
2) Inconsistent Data (suppose you forget to update Department Loca of HR in some rows)
3) Update / write operations → Extremely slow because you will have to update in multiple places (redundant).

Scanned with OKEN Scanner